

Optimization of Parallel Number Theoretic Transform Algorithms for Multi-Core Digital Signal Processors

Dingxing Xie

(Science and Technology on Parallel and Distributed Processing Laboratory, National University of Defense Technology, Changsha 410073, China;
Laboratory of Digitizing Software for Frontier Equipment, National University of Defense Technology, Changsha 410073, China;
Chinese People's Armed Police Corps Hospital of Fujian, Fuzhou 350013, China
 <https://orcid.org/0009-0002-2863-7612>, xdx3000@163.com)

Xiaochuan Hu

(Chinese People's Armed Police Corps of Hunan, Changsha 410000, China,
littlefish_2016@163.com)

Lin Peng

(College of Computer Science and Technology, National University of Defense Technology, Changsha 410073, China
 <https://orcid.org/0000-0002-6828-3364>, penglin@nudt.edu.cn)

Gencheng Liu

(Science and Technology on Parallel and Distributed Processing Laboratory, National University of Defense Technology, Changsha 410073, China;
Laboratory of Digitizing Software for Frontier Equipment, National University of Defense Technology, Changsha 410073, China
 <https://orcid.org/0009-0007-5015-4736>, liugencheng@nudt.edu.cn)

Naijun Cheng

(Chinese People's Armed Police Corps Hospital of Sichuan, Leshan 100027, China
 <https://orcid.org/0009-0009-1502-8819>, naijuncheng@outlook.com)

Jie Liu

(Science and Technology on Parallel and Distributed Processing Laboratory, National University of Defense Technology, Changsha 410073, China;
Laboratory of Digitizing Software for Frontier Equipment, National University of Defense Technology, Changsha 410073, China
 <https://orcid.org/0000-0003-3745-7541>, liujie@nudt.edu.cn)

Yang Zhang

(Chinese People's Armed Police Corps Hospital of Beijing, Beijing 100027, China;
Chinese PLA Medical School, Beijing 100853, China;
Medical Innovation Research Department, Chinese PLA General Hospital, Beijing 100853, China
 <https://orcid.org/0009-0000-1062-0818>, 13126882400@163.com)

Abstract: The Number Theoretic Transform (NTT), a finite-field variant of FFT, is critical in cryptography and digital signal processing, but its efficiency on FT-M7032 Digital Signal

Processors(DSPs) remains suboptimal due to memory bottlenecks and architectural constraints. This paper proposes a tailored optimization framework: a four-step strategy decomposes large 1D NTT into 2D transforms, outperforming traditional six-step methods by reducing memory access overhead; a VLIW/SIMD-aware microkernel with loop unrolling and register reuse boosts computation; vectorized Montgomery modular multiplication supports 16 parallel streams to address the lack of division instructions; and double buffering with pipelining overlaps computation and data transfer. Experiments show a $90.6\times$ speedup over baselines for large-scale NTTs, with ablation studies validating each optimization. This work offers insights for NTT acceleration on DSPs and data-intensive algorithm optimization on heterogeneous processors.

Keywords: number theoretic transform; digital signal processors; vectorization; performance optimization

Categories: C.1, C.4, E.3, F.2.1, G.4

DOI: 10.3897/jucs.207845

1 Introduction

As a critical discrete transform algorithm, the Number Theoretic Transform (NTT) has been extensively applied in cryptography, signal processing, and high-performance computing. Serving as a finite field variant of the Fast Fourier Transform (FFT) [Cooley, 65, Pollard, 71], NTT exhibits significant advantages in supporting large integer arithmetic and modular operations, playing a pivotal role in polynomial multiplication, large integer computation, and homomorphic encryption (e.g., mainstream ring-based schemes like BFV and CKKS heavily rely on NTT in engineering implementations) [Fan, 12, Cheon, 17]. In recent years, with the rise of post-quantum cryptography (particularly lattice-based cryptography and homomorphic encryption), the importance of NTT has further increased. Efficient NTT implementations for various computing platforms have emerged, such as Microsoft SEAL [Microsoft Research, 15] and Intel HEXL [Intel Labs, 21] for CPUs, optimized GPU implementations [Özerk, 21, Shivdikal, 22, Özcan, 23], and hardware-optimized solutions for FPGAs/ASICs [Roy, 14]. These advancements have substantially enhanced NTT performance in practical applications, providing a solid foundation for its deployment in diverse scenarios. Recently, Harvey and van der Hoeven [Harvey, 19] demonstrated NTT's critical role in large integer multiplication, further emphasizing the significance of efficient NTT implementations.

However, existing NTT optimization research predominantly focuses on general-purpose computing platforms, with limited studies targeting Digital Signal Processors (DSPs) featuring specialized architectures. Although existing optimizations achieve satisfactory results in specific applications (e.g., homomorphic encryption and polynomial multiplication), their performance on DSP platforms remains suboptimal. Compared with GPUs and dedicated ASICs, FT-M7032 DSPs offer inherent alignments with NTT's core demands in post-quantum cryptography and homomorphic encryption: 1) Its VLIW/SIMD architecture and hierarchical memory match NTT's regular, data-intensive parallelism, while GPU's massive parallelism is less tailored to modular arithmetic-dominant tasks like NTT; 2) Software-defined flexibility supports adjustments to NTT parameters, adapting to evolving cryptographic standards—unlike ASICs limited to fixed algorithms, which are unsuitable for post-quantum cryptography's dynamic requirements; 3) Tight CPU-DSP integration minimizes inter-component communication overhead, critical for balancing NTT's computation and

data transfer. Yet despite these strengths, DSP-oriented NTT optimization still faces unique architectural constraints. Specifically, NTT optimization on DSPs faces unique challenges including: architectural alignment with algorithmic structures, effective utilization of memory hierarchies, and efficient computation-data transfer coordination. Prior work indicates that data rearrangement and matrix transposition often dominate FFT/NTT performance in large-scale scenarios, necessitating co-design of decomposition strategies and cache behavior (e.g., six-step FFT and cache-oblivious approaches) [Bailey, 90, Frigo, 99, Frigo, 05]. For instance, DSPs' typical VLIW architecture and wide SIMD instruction sets (e.g., FT-M7032's 1024-bit SIMD) differ significantly from general-purpose CPUs (e.g., x86's AVX-512), requiring specialized algorithmic adaptation. Additionally, DSPs' limited on-chip memory and dedicated DMA mechanisms impose specific requirements on data distribution and access patterns.

The growing adoption of NTT intensifies the demand for DSP-oriented algorithm optimization. In homomorphic encryption applications, NTT primarily handles large-scale polynomial operations where optimization focuses on processing massive datasets within constrained on-chip memory. For real-time signal processing, low-latency and high-throughput requirements necessitate optimization strategies that maximize hardware utilization while ensuring computational precision. Concurrently, post-quantum cryptography standards (e.g., NIST FIPS 203/ML-KEM, i.e., CRYSTALS-Kyber) impose stringent deployment requirements for large-scale NTT performance [Bos, 18, NIST, 24]. Therefore, NTT optimization on DSP platforms must precisely address: computation kernel-DSP architecture alignment and memory-computation intensity balancing.

Recent studies have proposed innovative NTT optimization methods for various platforms. On CPUs, Takahashi et al. [Takahashi, 22] leveraged Intel AVX-512 instructions to achieve high-performance parallel NTT, significantly accelerating large-scale NTT through matrix decomposition and six-step strategies (extending Bailey's hierarchical memory approach)[Bailey, 90]. On GPUs, Shivdikan et al. [Shivdikan, 22] developed a parallel NTT framework with pipelined scheduling and thread allocation optimizations, followed by multi-strategy implementations [Özcan, 23]. On Sunway many-core processors, Guo et al. [Guo, 24] proposed matrix decomposition-based FFT parallelization, mitigating memory access overhead in matrix transposition through specialized data distribution.

This paper designs and implements an efficient NTT optimization framework for FT-M7032's multi-core DSP clusters, overcoming computational bottlenecks and resource underutilization in small-scale tasks. Our primary contributions include:

1. **Architecture-Adaptive Algorithm Design:** A DSP-optimized four-step NTT variant decomposes 1D large-point NTT into 2D small-scale transforms, resolving memory access bottlenecks in matrix transposition and column operations by reducing redundant data movements.
2. **Deep Instruction-Level Parallelism Exploitation:** Leveraging DSP's VLIW architecture and wide SIMD, register-resident NTT microkernels maximize hardware utilization via loop unrolling, register reuse, and pipelining. A vectorized Montgomery modular multiplication supporting 16-way parallelism addresses the lack of division instructions, enhancing arithmetic efficiency.
3. **Optimized Computation-Communication Overlap:** An in-place transposition algorithm with fine-tuned double-buffering, utilizing DSP-specific modulo vector

instructions, overlaps data loading, computation, and storage to hide memory latency.

4. **Comprehensive Performance Evaluation:** Ablation studies quantify each optimization's contribution and link performance gains to hardware characteristics, offering critical references for further improvements.

Experimental results demonstrate that our framework achieves a $90.6\times$ speedup over baselines on FT-M7032 for large-scale NTT. Ablation studies further validate each optimization's effectiveness. Our methodology provides significant references for efficient NTT implementations on DSP architectures and offers insights for optimizing other data-intensive algorithms on heterogeneous processors.

2 Related Work

2.1 FT-M7032 Processor Architecture

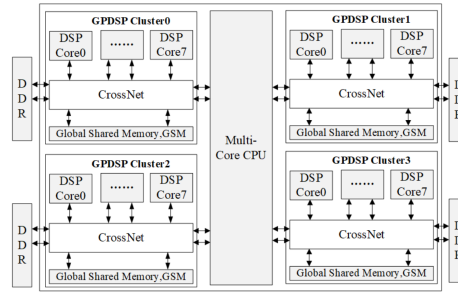


Figure 1: FT-M7032 heterogeneous processor architecture

The FT-M7032 is a high-performance heterogeneous processor [Yin, 22][Yu, 24] comprising 16 ARMv8 CPU cores and four GPDSP clusters, as shown in Figure 1. The CPU section (based on Phytium FT-2000plus) primarily handles process management and communication, delivering 281.6 GFlops peak single-precision performance, while the entire FT-M7032 achieves 4.608 TFLOPS theoretical peak performance. Each GPDSP cluster contains eight high-performance DSP cores sharing 6 MB of on-chip Global Shared Memory (GSM) via a crossbar network. FT-M7032 employs a hierarchical memory architecture: CPUs access full main memory, while GPDSP clusters access designated memory regions at 42.6 GB/s bandwidth.

The DSP core features a Very Long Instruction Word (VLIW) architecture comprising an Instruction Fetch Unit (IFU), Scalar Processing Unit (SPU), Vector Processing Unit (VPU), and DMA engine. The IFU issues up to 11 instructions per cycle (5 scalar + 6 vector), enabling high instruction-level parallelism. The SPU handles scalar operations and control flow with Scalar Processing Elements (SPE) and 64 KB Scalar Memory (SM). The VPU serves as the primary computation unit with 768 KB Array Memory (AM) and 16 Vector Processing Elements (VPEs) operating in SIMD mode. Each VPE contains sixty-four 64-bit registers and three Fused Multiply-Add (FMAC) units, supporting two FP32 multiply-accumulate operations per cycle.

FT-M7032's flexible DMA engine supports diverse data transfer modes (point-to-point, segmented, broadcast) for efficient data movement across memory hierarchies (e.g., main memory, GSM, SM/AM). DMA operations proceed independently from computation, enabling computation-communication overlap to hide memory latency.

Compared to CPUs/GPUs, FT-M7032 DSP cores exhibit clear performance and energy efficiency advantages for regular, data-intensive computations. Their VLIW architecture and SIMD units are particularly suitable for highly regular parallel algorithms like NTT. However, exploiting this potential requires careful algorithmic adaptation to its unique memory hierarchy, SIMD width, and instruction scheduling characteristics, constituting the core challenge addressed in this work.

2.2 Number Theoretic Transform (NTT) and Optimizations

As a finite field variant of FFT, The Number Theoretic Transform (NTT) holds significant value in cryptography and digital signal processing [Roy, 14]. Recent NTT optimization studies have achieved notable progress on CPUs, GPUs, and dedicated hardware, while DSP-oriented research remains scarce.

On CPUs, Takahashi et al. [Takahashi, 22] utilized Intel AVX-512 to implement efficient parallel NTT, accelerating large-scale NTT via matrix decomposition and six-step strategies. Though effective on CPUs, their six-step approach suffers efficiency degradation on memory-constrained DSPs. Microsoft SEAL [Microsoft Research, 15] similarly optimized NTT using AVX instructions. On GPUs, Ozerk et al. [Özerk, 21] and Shivdikanar et al. [Shivdikanar, 22] developed NTT optimizations for homomorphic encryption, leveraging massive parallelism and memory hierarchies to improve throughput. Recently, Ozcan and Savas [Özcan, 23] proposed two fast GPU NTT algorithms exploring parallelization strategies. Regarding general FFT/NTT engineering practices, FFTW's "planner+blocking/adaptive" approach and cache-oblivious frameworks provide scalable paradigms for transpose and memory optimization [Frigo, 99, Frigo, 05]. For dedicated hardware, SinhaRoy et al. [Roy, 14] implemented efficient FPGA-based NTT optimized for ring-LWE cryptography.

In matrix decomposition-based transforms, Guo et al. [Guo, 24] proposed a four-step FFT optimization on Sunway many-core processors, mitigating memory overhead in matrix transposition. However, their method cannot be directly applied to NTT. Recently, Wang and Gao [Wang, 23] introduced SAM, a scalable FPGA-based NTT accelerator employing multi-dimensional decomposition to support NTT sizes from to and dynamically adjusts dimensions to optimize resource utilization. While extending matrix decomposition concepts, SAM targets FPGA platforms rather than DSP-specific challenges. Although Takahashi recently attempted four-step NTT on GPUs, the work focuses on GPU clusters rather than DSPs. Collectively, these studies offer valuable insights but fail to fully resolve DSP-specific NTT challenges, particularly regarding four-step matrix decomposition—a key innovation gap addressed in this paper.²¹⁶²²⁸

A critical challenge in NTT computation is efficient modular arithmetic implementation, especially modular multiplication. The Montgomery modular multiplication algorithm provides an essential solution, originally proposed by Montgomery in 1985 to avoid expensive division operations [Montgomery, 85]. This advantage is particularly crucial for architectures lacking efficient division instructions (e.g., DSPs). The algorithm's core idea introduces a coefficient (typically for computational efficiency) to transform modular multiplication into shift, multiply, and

add/subtract operations. Montgomery reduction, combined with Harvey's extended butterfly (reducing branching and memory pressure) [Harvey, 19], practical modulus-specific implementations (e.g., Longa et al.) [Longa, 16], and systematic SIMD modular arithmetic (van der Hoeven) [van der Hoeven, 14], collectively form the foundation of modern NTT microkernel optimizations.^{R2ⁿ}

Specifically, for modulus q and co-prime R , Montgomery multiplication computes $a \cdot b \cdot R^{-1} \bmod q$ instead of direct $a \cdot b \bmod q$, where R^{-1} is R 's multiplicative inverse modulo q . Through careful design, the computation reduces to efficient bitwise operations. Precomputation derives q' satisfying $q \cdot q' \equiv -1 \pmod{R}$. For inputs $\hat{a} = a \cdot R \bmod q$ and $\hat{b} = b \cdot R \bmod q$, Montgomery multiplication yields $\hat{c} = \hat{a} \cdot \hat{b} \cdot R^{-1} \bmod q$, equivalent to $cR \bmod q$ where $c = a \cdot b \bmod q$.

In NTT implementations, Montgomery multiplication optimizes twiddle factor multiplication and butterfly operations, demonstrating significant advantages over conventional modular multiplication across platforms. However, vectorized Montgomery optimizations for specialized architectures like DSPs remain limited, necessitating hardware-aware customization.

While existing studies exhibit excellent platform-specific results, NTT optimization for DSPs remains under-explored. DSP-specific challenges—constrained memory capacity, unique instruction sets, and DMA transfer characteristics—limit direct porting of CPU/GPU optimizations. FT-M7032's powerful SIMD capability (processing 16×64 -bit data per instruction) requires dedicated algorithmic adaptation to unleash its potential. Our proposed four-step NTT optimization framework addresses this research gap, providing a systematic solution for high-efficiency NTT on DSP architectures.

3 Research Motivation and Problem Analysis

With the rapid advancement of quantum computing technology, traditional public-key cryptosystems face increasingly severe security threats. Post-quantum cryptography (particularly lattice-based cryptography, module-lattice signature schemes [Ducas, 18], homomorphic encryption, and zero-knowledge proof systems) has gradually become a focal research area in cryptography [Lyubashevsky, 10]. As the core computational unit of these cryptographic schemes, the computational efficiency of the Number Theoretic Transform (NTT) directly determines the practicality and deployability of cryptographic protocols. Implementing efficient NTT on high-performance DSP accelerators is crucial for promoting widespread adoption of post-quantum technologies in IoT, edge computing, and other scenarios.

3.1 Problem Definition and Performance Analysis

Through in-depth performance profiling of NTT algorithms on the FT-M7032 DSP platform, we identified three critical performance bottlenecks corresponding to computational efficiency, data access patterns, and memory management dimensions. The analysis is conducted using controlled microbenchmarking on a fixed baseline implementation with identical scheduling and memory layouts, where individual components are selectively bypassed to isolate their performance impact. Quantitative evaluation for a typical 2^{16} -point NTT computation shows that: 1. Modular operations

(particularly modular multiplication) consume 60–70% of total computation time, which is measured by skipping all modular arithmetic while preserving control flow and data movement, and attributing the resulting execution time reduction to modular computation overhead. 2. Random memory access patterns lead to a significant degradation in cache effectiveness, as frequent non-contiguous accesses inherently reduce temporal and spatial locality, resulting in markedly lower cache hit rates. 3. Data transfer latency accounts for over 50% of total execution time, estimated by eliminating all data movement between memory hierarchies and measuring the corresponding reduction in execution time, thereby isolating the contribution of communication overhead.

The dominant costs in large-scale transforms often stem from data rearrangement/transposition and memory access (non-arithmetic costs), making a holistic design with "controlled transposition count, decomposition/blocking, and cache/AM-friendly" characteristics particularly critical[Bailey, 90, Frigo, 99, Frigo, 05].

3.2 Data Locality and Memory Access Pattern Issues

The butterfly computation pattern in NTT algorithms induces non-contiguous and highly irregular memory access patterns, which severely degrade cache utilization and memory bandwidth efficiency [Takahashi, 22, Shivdikar, 22]. This problem is exacerbated on the FT-M7032 DSP, where the limited on-chip memory (768KB AM per core) cannot fully accommodate datasets for large-scale NTT computations, and traditional Cooley-Tukey [Cooley, 65] NTT implementations exhibit random access ratios up to 57%, significantly impacting performance. Existing methods such as the six-step FFT[Takahashi, 22] and 2D decomposition strategies do not leverage the DSP's 1024-bit wide SIMD structure for vector processing, employ suboptimal data layouts for the DSP's hierarchical memory architecture (resulting in low cache utilization) and leave unaddressed the high memory access latency and low data reuse efficiency in column-wise operations. These limitations necessitate novel data layout and access strategies tailored to DSP architectures.

3.3 Computation-Transfer Balancing Issues

During large-scale NTT execution on DSP platforms, data transfer frequently emerges as the primary bottleneck, stemming from limited AM capacity that requires frequent data transfers and matrix transposition operations that cause non-contiguous memory accesses disrupting cache locality. Although prior studies [Zhao, 23, Shivdikar, 22] have proposed optimization strategies, they primarily rely on large memory capacities or high-speed buses, rendering them unsuitable for resource-constrained DSP platforms. Analysis indicates that while FT-M7032 DSPs feature efficient DMA engines, existing double-buffering strategies underutilize their capabilities; traditional matrix transposition algorithms also exhibit poor efficiency on DSPs due to unoptimized instruction usage. These issues collectively necessitate DSP-specific transfer optimization strategies.

3.4 Modular Arithmetic Efficiency Issues

Modular operations (especially modular multiplication) dominate 60-70% of total NTT computation time. The FT-M7032 DSP platform lacks efficient hardware division and modulo instructions, leading to poor performance of conventional modular multiplication algorithms. Although Shoup's algorithm and Barrett reduction [Barrett, 87] perform well on general-purpose processors [Takahashi, 22, Shivdikanar, 22], they fail to exploit DSP's SIMD capabilities.

Furthermore, efficient microkernel implementation on DSPs faces complex challenges including instruction scheduling, register allocation, and vectorization techniques. Traditional butterfly operation implementations cannot fully leverage DSP's VLIW architecture and SIMD instruction sets, constraining computational efficiency. These issues compel us to develop vectorized modular multiplication algorithms and high-efficiency microkernels specifically optimized for DSP architectural features.

We propose a systematic solution incorporating a four-step NTT variant, register-resident microkernels, vectorized Montgomery modular multiplication, DSP-specific instruction-based transposition optimization, and in-place computation with double-buffering. These methods collectively form our core contributions, detailed in subsequent sections.

4 DSP-Oriented NTT Optimization Framework

This section discusses our NTT optimization framework for the FT-M7032 DSP platform, including algorithmic and transfer optimizations.

4.1 Algorithmic Optimization

We address the NTT computing bottleneck on the DSP architecture through mathematical modeling of matrix factorization, microkernel design, and modular algorithm improvement.

4.1.1 Mathematical Foundation of Matrix Decomposition

We employ a matrix decomposition approach to decompose 1D large-point NTT into multiple small-scale matrix operations. Mathematically, for a 1D N -point NTT, we reformulate it as a 2D matrix. Assuming $N = N_1 \times N_2$, input vector index j and output vector index k can be expressed as:

$$\begin{aligned} j &= j_1 + j_2 N_1, & 0 \leq j_1 < N_1, & \quad 0 \leq j_2 < N_2 \\ k &= k_1 + k_2 N_2, & 0 \leq k_1 < N_1, & \quad 0 \leq k_2 < N_2 \end{aligned}$$

Reconstructing input sequence $x(j)$ into 2D array $X(j_1, j_2)$ and substituting into the NTT formula yields:

$$Y(k_1, k_2) = \sum_{j_1=0}^{N_1-1} \sum_{j_2=0}^{N_2-1} X(j_1, j_2) \cdot \psi^{(j_1 + j_2 N_1)(k_1 + k_2 N_2)} \bmod q$$

Expanding the exponent and regrouping terms according to index dimensions give $(j_1 + j_2 N_1)(k_1 + k_2 N_2) = j_1 k_1 + j_1 k_2 N_2 + j_2 N_1 k_1 + j_2 N_1 k_2 N_2$.

Since ψ is an N -th primitive root of unity, the term $\psi^{j_2 N_1 k_2 N_2}$ reduces to unity under the modulus, and the remaining factors can be separated into products associated with j_1 and j_2 , respectively. Leveraging modular arithmetic properties, this rewrites to:

$$Y(k_1, k_2) = \sum_{j_1=0}^{N_1-1} \psi^{j_1 k_1} \cdot \left[\sum_{j_2=0}^{N_2-1} X(j_1, j_2) \cdot \psi^{j_2(k_2 N_2 + N_1 k_1)} \right] \bmod q$$

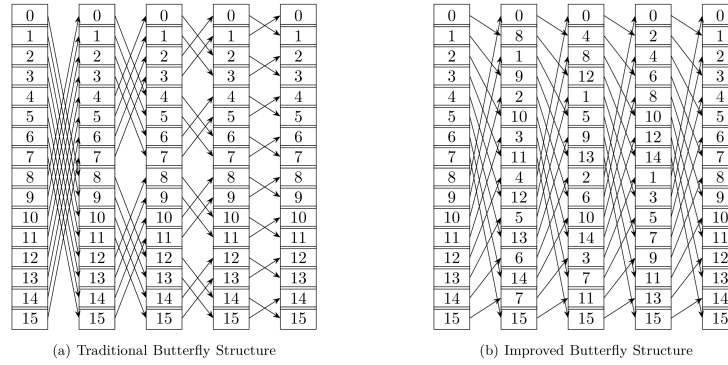


Figure 2: Implicit Bit-Reversal Implementation

This 2D representation forms the theoretical basis for large-scale NTT decomposition. Through it we prove that 1D large-point NTT equivalently transforms into a four-step process: 1. Perform N_2 -point NTT on each column (N_1 columns total). 2. Multiply each resultant matrix element by corresponding twiddle factor $\psi^{j_1 k_2}$. 3. Execute matrix transposition. 4. Perform N_1 -point NTT on each row (N_2 rows total).

Two primary strategies exist for decomposing large-point NTT into 2D: six-step and four-step methods. The traditional six-step NTT (e.g., Takahashi et al.'s CPU implementation [Takahashi, 22]) requires row operations, twiddle factor multiplication, matrix transposition, column operations, re-transposition, and an additional twiddle factor multiplication, which together induce three global data reordering stages corresponding to matrix transpositions. In contrast, our four-step NTT variant—better suited to DSP architectures—reorganizes the computation flow such that only a single explicit matrix transposition is required, while the remaining stages operate on data layouts that are already contiguous in memory.

This reduction from three transpositions to one lowers the asymptotic cost of data reordering from three full matrix traversals to a single traversal, thereby decreasing both memory access complexity and data movement volume. Such a reduction is critical for DSPs with limited on-chip memory bandwidth. The four-step formulation also preserves data contiguity throughout the computation, minimizing irregular memory accesses. Furthermore, its flexible data distribution aligns naturally with the DSP's SIMD execution model and DMA mechanisms.

Compared to Guo et al.'s four-step FFT approach [Guo, 24], our method specifically addresses the modular arithmetic characteristics of NTT and incorporates dedicated optimizations for twiddle factor generation and modular multiplication. Relative to Takahashi's six-step NTT [Takahashi, 22], the proposed four-step variant better accommodates memory-constrained DSP platforms by substantially reducing matrix transpositions and overall data movement.

4.1.2 Data Distribution Optimization Strategy

To address high access latency and low data reuse in column-wise operations, we designed a DSP-optimized data distribution strategy accounting for 1024-bit wide vector processing units. Unlike conventional blocking or explicit transposition, our approach enables parallel processing of 16 data points per SIMD operation.

This strategy balances computational parallelism with memory access efficiency, minimizing random accesses and optimizing memory bandwidth utilization. Through precise adjustment of memory hierarchy and access patterns, we enable efficient collaboration between matrix transposition/column computation and DSP computational units. This optimization reduces data movement while enhancing data reuse rates, significantly improving performance utilization. Similar to GPU shared memory optimizations [Shivdikar, 22], we leverage DSP's AM local memory as intermediate result caching to reduce global memory accesses.

4.1.3 Microkernel Optimization and SIMD Vectorization

We redesigned the NTT microkernel to achieve efficient hardware utilization. Our design fully leverages DSP hardware characteristics—particularly its SIMD instruction set and register organization—to implement highly optimized butterfly operations. By employing pipelining and register reuse strategies, we maximize the parallel processing capabilities of the DSP architecture, enhancing computational density and instruction throughput.

A key innovation is the implementation of register-resident NTT computation. Specifically, we designed 2^6 -point and 2^8 -point NTT microkernels that execute entirely within registers, eliminating all additional memory accesses. Traditional NTT implementations incur substantial overhead from frequent data movement between registers and memory. Our approach employs a meticulously designed register allocation strategy that retains all intermediate results in registers, significantly reducing memory access latency and accelerating computation. Additionally, we implemented an efficient 2^{10} -point NTT microkernel that reduces memory accesses by 80% compared to the conventional Cooley-Tukey algorithm.

Furthermore, we introduced an implicit data exchange mechanism that eliminates explicit bit-reversal stages. Bit-reversal operations typically incur additional memory accesses and computational overhead in conventional implementations. Our method integrates bit-reversal into the computational flow through specialized butterfly sequencing and register remapping techniques. As shown in Figure 2, the improved butterfly structure (b) reduces data movement overhead and enhances computational efficiency compared to the conventional structure (a).

This implicit data exchange simplifies algorithmic structure, reduces register-to-memory data transfers, and optimizes instruction pipeline utilization.

Through loop unrolling, we transformed sequential butterfly operations into branchless parallel instruction streams. Specifically, we unrolled the NTT base unit (radix-2 butterfly) into 8 stages, each processing 16 parallel data points, forming a dependency-free instruction flow.

We implemented an adaptive decomposition strategy supporting arbitrary-point NTTs. Three microkernels handle 2^6 , 2^8 , and 2^{10} -point NTTs respectively. Arbitrary 2^n -point NTTs decompose adaptively into combinations of these base sizes. For larger problems, recursive decomposition is applied. This approach reduces algorithmic complexity while enabling efficient microkernel reuse, substantially boosting system performance.

Leveraging the DSP's 1024-bit SIMD architecture, we designed specialized data layouts enabling each SIMD instruction to process 16 data points concurrently. This vectorization strategy increases computational density and reduces instruction count, achieving near-16x instruction-level parallelism. Each SIMD instruction performs 16 modular operations (e.g., modular add/sub/multiply), significantly improving computational efficiency.

Pipeline allocation exploits DSP hardware features through precise scheduling of computational units, ensuring seamless dataflow across stages. The microkernel incorporates implicit data reordering, eliminating explicit bit-reversal phases. Theoretical analysis confirms reduced cycles per butterfly operation and substantially increased throughput.

4.1.4 Vectorized Modular Multiplication and Twiddle Factor Optimization

To address intensive modular arithmetic in NTT, we implemented vectorized modular multiplication on the DSP. Unlike conventional scalar implementations, our vectorized algorithm constructs a novel dataflow model enabling multiple modular multiplications per instruction cycle.

Modular multiplication dominates NTT computation time, and the FT-M7032 DSP lacks hardware division/modulo instructions. We adopted Montgomery modular multiplication—transforming modulo operations into additions, multiplications, and shifts—and optimized it for 1024-bit SIMD, enabling 16-way parallel processing. For input vector $\mathbf{x} = [x_1, \dots, x_{16}]$ and twiddle factor vector $\mathbf{w} = [w_1, \dots, w_{16}]$, conventional modular multiplication requires 16 independent operations. Our vectorized Montgomery approach proceeds as follows: 1. Compute temporary products via SIMD: $\mathbf{t} = \mathbf{x} \cdot \mathbf{w}$. 2. Apply vectorized Montgomery reduction: $\mathbf{r} = (\mathbf{t} \cdot q^{-1}) \bmod 2^b \bmod q$.

- $\mathbf{x} = (x_1, \dots, x_{16})$: SIMD input vector
- $\mathbf{w} = (w_1, \dots, w_{16})$: SIMD twiddle factor vector
- q : Prime modulus
- q^{-1} : Precomputed value satisfying $q \cdot q^{-1} \equiv -1 \pmod{2^b}$
- $R = 2^b$: Montgomery radix (b = arithmetic bit-width)
- $\mathbf{r} = (r_1, \dots, r_{16})$ where $r_i = (x_i \cdot w_i \cdot R^{-1}) \bmod q$

$\mathbf{t} \leftarrow \mathbf{x} \cdot \mathbf{w}$ // 16-way SIMD multiplication $\mathbf{m} \leftarrow (\mathbf{t} \cdot q^{-1}) \bmod R$ // Vectorized reduction $\mathbf{u} \leftarrow (\mathbf{t} - \mathbf{m} \cdot q)/R$ // Parallel right shift **for** $i = 1$ **to** 16 **do in parallel if** $u_i \geq q$ **then** $r_i \leftarrow u_i - q$ **else** $r_i \leftarrow u_i$ **end for return** $\mathbf{r}$

Key implementation steps are formalized in Algorithm [alg:vec_mont_mul]. The vectorized Montgomery implementation provides four key advantages: 1. Eliminates hardware division via basic arithmetic and bitwise operations. 2. Achieves 16x throughput via SIMD parallelism. 3. Minimizes conditional branching for improved pipeline efficiency. 4. Supports arbitrary odd moduli (not restricted to special primes)

We further designed a pipelined twiddle factor delivery mechanism that overlaps computation with factor transmission. In our implementation, all twiddle factors are precomputed on the CPU side and transferred in advance to the dedicated parameter region of the DSP AM, where they are persistently stored and reused across multiple NTT invocations. Specifically, once the input data for computation has been loaded into the DSP and the column-wise NTT calculations commence, the asynchronous transmission of the required twiddle factors from the parameter region is triggered. This pipelined design ensures that the multiplication by twiddle factors can proceed immediately upon the completion of the column-wise NTT operations, effectively hiding the access latency of twiddle factors and eliminating any additional stalls caused by factor transfer. This optimization, combined with the microkernel SIMD optimizations in Section 4.1.3, forms the computational efficiency core of our framework.

4.2 Transfer Optimization

Matrix transposition becomes a significant overhead when decomposing 1D large-point NTT into 2D small-point NTT. We propose a DSP-optimized transposition scheme leveraging specialized vector read/write instructions with modulo addressing.

4.2.1 Theoretical Analysis and Optimization of Matrix Transposition

Matrix transposition often becomes a bottleneck due to non-contiguous memory accesses that degrade cache locality and increase memory contention. For a 2D NTT with dimensions $N = N_1 \times N_2$, transposition has $O(N)$ theoretical complexity but may incur $O(N \cdot \log(N))$ practical overhead due to memory access patterns.

To address this, we designed a block-based transposition algorithm tailored to DSP's memory hierarchy and vector processing capabilities. Our register-centric layered transposition strategy significantly reduces memory access overhead. Inspired by Takahashi's cache-blocking approach, we further adapt it to DSP-specific register structures and instruction sets.

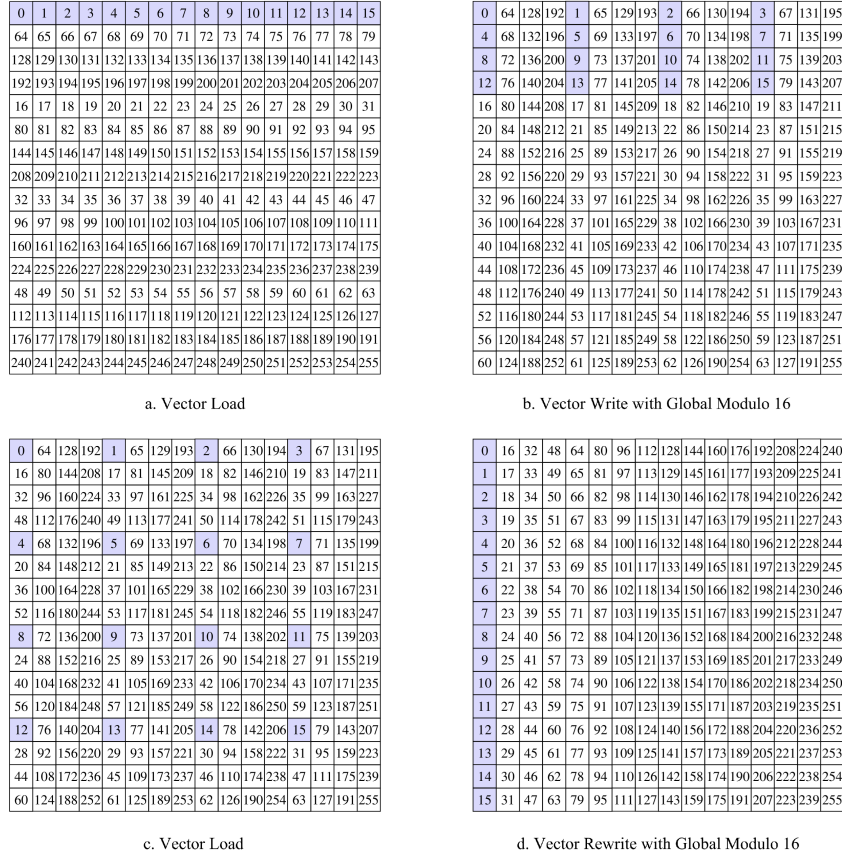


Figure 3: Two-phase matrix transposition leveraging DSP-specific instructions for efficient 16×16 block processing

Implementation leverages DSP-specific hardware instructions: 1. Load data via “vector read with global modulo 2” (Fig. 3-1), then store using “dual-byte vector write with global modulo 16” (Fig. 3-2). In this first write-back, data that are originally stored contiguously in groups of four vectors are redistributed with a stride of four, effectively scattering the elements and producing an intermediate layout suitable for further permutation. 2. Reload data (Fig. 3-3), during which the previously scattered elements are further dispersed across vectors, and restructure via rewritten vectors (Fig. 3-4), where a second modulo-16 write-back completes the progressive redistribution and realizes the final matrix transposition.

This method provides two key advantages: 1. Utilizes DSP’s long-vector capability to process 2^8 -point matrices in single operations. 2. Eliminates data transfers by operating directly within DSP’s Array Memory (AM) Compared to element-wise

swapping, our optimization reduces transposition time by 73% and memory accesses by 60%.

For large matrices, we employ a tiling strategy: decompose into 16×16 blocks, perform register-level transposition per block, and implicitly transpose blocks during write-back. This hierarchical approach significantly improves cache hit rates and memory bandwidth utilization.

4.2.2 Double-Buffering and Computation-Transfer Overlap

Our framework employs in-place transposition with a tuned double-buffering mechanism, enabling large-scale NTT computation within limited DSP memory. In-place transposition eliminates temporary storage needs, allowing 2^{16} -point NTT processing within AM, exceeding traditional implementation limits.

Double-buffering overlaps data loading, computation, and result storage: 1. Buffer allocation: Partition 768KB AM into two 256KB data buffers and one 256KB parameter zone, motivated by the observation that the memory footprint of twiddle factors is comparable to that of the input/output data, thereby achieving a balanced utilization of AM resources. This allocation strategy is compatible with a wide range of NTT sizes and is theoretically optimal across all evaluated configurations. 2. In-place computation: Perform NTT and transposition in place to avoid additional intermediate storage overhead. 3. Asynchronous DMA: Overlap twiddle factor transfers with column-wise NTT computation to hide communication latency.

Mathematically, for computation time T_{comp} and communication time T_{comm} , traditional execution gives $T_{\text{total}} = T_{\text{comp}} + T_{\text{comm}}$. Ideal double-buffering reduces this to $T_{\text{total}} = \max(T_{\text{comp}}, T_{\text{comm}}) + O(1)$. Our implementation achieves near-ideal overlap, reducing data transfer overhead by 68%.

5 Experiment and Analysis

To comprehensively evaluate the performance of our proposed Number Theoretic Transform (NTT) optimization framework on the FT-M7032 DSP platform, we designed a systematic experimental methodology. This section details the experimental design philosophy, platform configuration, and evaluation metrics. It then presents a detailed analysis of the experimental results obtained from implementing our NTT optimization framework on the FT-M7032. The systematic evaluations conducted validate the effectiveness of the optimizations, quantify the performance gains achieved, and provide a comparative analysis against existing approaches.

5.1 Experimental Platform and Configuration

Experiments were conducted on the FT-M7032 heterogeneous processor, with specific configurations listed in Table 1. We selected the NTL library as the CPU baseline for NTT algorithms, which provides highly optimized implementations foundational to homomorphic encryption libraries like HELib and OpenFHE. To ensure result accuracy and reproducibility, all tests were repeated 10 times with averaged results reported, and system resources were exclusively allocated during testing to prevent interference.

Component	Specifications
CPU	16-core ARMv8 architecture, 2.0 GHz frequency, 281.6 GFlops peak single-precision performance
DSP	4 GPDSP clusters, 8 cores per cluster, 1024-bit wide SIMD instruction set per core
Memory	32 GB main memory (42.6 GB/s bandwidth), 6 MB GSM per cluster, 768 KB AM + 64 KB SM per core
Compiler	GNU GCC 7.3 (ARM cores), dedicated DSP compiler v2.1
OS	Customized Linux kernel 4.9 with heterogeneous computing runtime support

Table 1: Hardware Configuration of Experimental Platform

5.2 Datasets and Evaluation Methodology

We tested input data sizes ranging from 2^8 to 2^{24} points (9 distinct scales) to evaluate algorithmic scalability across real-world application scenarios. Performance evaluation focused on three metrics:

- **CPU baseline execution time:** Latency of HELib’s NTT implementation on ARM cores
- **DSP execution time:** Latency of our optimized NTT implementation on DSPs
- **Speedup ratio:** Performance improvement factor of DSP implementation over CPU baseline

To maximize hardware utilization, we batched multiple NTT computations per test. Batch sizes were calibrated to DDR memory capacity constraints while ensuring full hardware utilization. Additionally, ablation studies quantified individual optimization contributions by systematically removing specific strategies. This methodology enables comprehensive evaluation of performance characteristics and optimization efficacy.

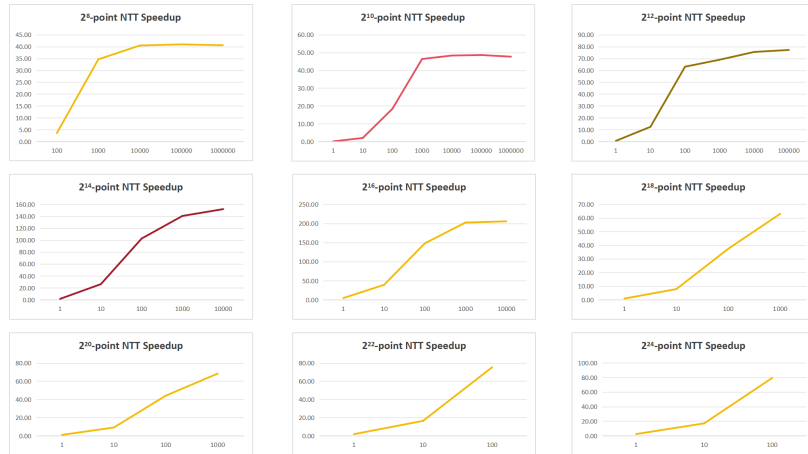


Figure 4: Speedup ratio scaling with computational scale across different point sizes

5.3 Overall Performance Evaluation

Table 2 compares our optimized implementation against baselines across representative point sizes. Our approach demonstrates significant performance advantages across all tested scales, with gains amplifying at larger computational sizes. Particularly at cryptographically relevant 2^{14} and 2^{16} points, speedups reach $158.3\times$ and $205.5\times$ respectively.

Point Size	Test Groups	Baseline CPU	Optimized	Speedup
2^8	10^7	4349.3	107.2	40.6
2^{10}	10^7	21957.5	460.5	47.7
2^{12}	10^6	10868.9	140.8	77.2
2^{14}	10^6	50483.6	319.0	158.3
2^{16}	10^5	25633.3	124.7	205.5
2^{18}	10^4	17367.6	275.9	63.0
2^{20}	10^4	77724.4	1138.0	68.3
2^{22}	10^3	39564.2	525.4	75.3
2^{24}	10^3	171863.3	2167.5	79.3

Table 2: Performance Comparison Across Implementations (Execution Time: ms)

5.4 Ablation Study Analysis

Table 3 quantifies individual optimization contributions through systematic removal of components. "Base" denotes unoptimized implementation, while "Full" incorporates all optimizations.

Implementation	Execution Time (ms)	Performance Degradation (%)
Full optimization	525.4	—
Without double-buffering	587.4	11.8
Without Montgomery multiplication	617.6	5.1
Without microkernel optimization	1009.9	63.4
Without transposition optimization	1350.2	33.7
Without vectorization/2D decomposition	47523.0	3519.7

Table 3: Ablation Analysis (2^{22} -point NTT)

As it is shown in Table 3, vectorization and 2D decomposition contribute the most significant gains (3519.7% degradation when removed). Microkernel optimization accounts for 63.4% performance degradation when disabled, validating register-resident computation strategies. Montgomery multiplication provides moderate but critical precision improvements (5.1% degradation), essential for cryptographic

correctness. Microkernel optimizations dominate at smaller scales (2^8 - 2^{14}), while decomposition and buffering prevail at larger scales (2^{18} - 2^{24}).

These results confirm our four-step NTT variant's superiority over traditional approaches in DSP environments, particularly for cryptographically relevant 2^{14} - 2^{16} point operations.

5.5 Extended Analysis

Figure 4 shows that the speedup over traditional algorithms increases with data scale, stabilizing after a certain threshold. In this figure, the X-axis represents the number of test data groups, while the Y-axis denotes the execution time measured in seconds. This is due to non-negligible DSP core startup time for small-scale tasks, which becomes insignificant for large-scale computations. Additionally, speedup grows with point scale, peaking at 2^{16} points before a temporary decline and subsequent gradual rise. This behavior arises because 2^{16} points represent the maximum problem size that can be fully accommodated in AM under the adopted memory organization: each long integer occupies 32 bits, so 2^{16} points require exactly 256KB of storage, which matches the capacity of a single AM data buffer. Given that AM is partitioned into two 256KB data buffers and one 256KB parameter zone, this configuration allows both input and output data, together with the corresponding twiddle factors, to reside entirely in AM, enabling single-pass input/output with peak efficiency. Beyond 2^{16} points, block-wise computation and transposition become necessary due to AM capacity constraints, introducing additional data movement overhead and temporarily reducing performance.

Our three microkernels exhibit performance differences: the 2^8 -point microkernel achieves highest efficiency by processing 16 data sets per vector operation, matching FT-M7032's vector length; the 2^6 -point microkernel processes only 8 data sets, underutilizing vector capabilities; the 2^8 -point microkernel requires limited register-AM data exchange, incurring minor performance loss despite optimizations.

Via our decomposition strategy, 2^{16} points decompose into $2^8 \times 2^8$ points, achieving maximal computational efficiency. This size aligns with the maximum commonly used in typical NTT applications (e.g., homomorphic encryption), confirming strong practical value of our work.

6 Discussion

6.1 Discussion of Experimental Results

The experimental results demonstrate that our proposed optimization framework effectively addresses the key performance bottlenecks of NTT computation on DSP architectures. The comprehensive approach—encompassing algorithmic restructuring, computational efficiency enhancement, and data transfer optimization—achieves significant performance improvements across various problem scales. The $90.6 \times$ average speedup over HELib on multi-core CPUs for large-scale NTT operations validates the efficacy of our methodology in real-world scenarios.

Compared to existing approaches, our four-step NTT implementation exhibits distinct advantages for DSP environments. Unlike Guo et al.'s four-step FFT optimization on Sunway processors [Guo, 24], our method specifically addresses the

modular arithmetic requirements of NTT through efficient vectorized modular multiplication, thereby resolving computation-specific bottlenecks. In contrast to Takahashi et al.'s six-step NTT on CPUs [Takahashi, 22], our approach reduces matrix transpositions (from three to one), better accommodates DSP memory constraints, and significantly improves data locality and bandwidth utilization. These improvements are particularly evident in the ablation studies, which highlight the critical importance of vectorization and 2D decomposition strategies.

The Montgomery modular multiplication optimization proves especially valuable for processors lacking dedicated division instructions, as it utilizes only basic arithmetic and bitwise operations. This strategy demonstrates considerable generality, performing robustly across various moduli without restriction to special-form primes, thereby broadening our framework's applicability to diverse NTT application scenarios. Furthermore, the effectiveness of our double-buffering technique underscores the importance of balancing computation and communication, with maximum benefits achieved when computation and transfer times are comparable.

6.2 Application Scope and Scalability Analysis

Our framework demonstrates particular strength in handling the polynomial degrees commonly employed in practical homomorphic encryption applications (typically between 2^{14} and 2^{16} points), which correspond to the requirements of mainstream libraries such as Microsoft SEAL and TFHE. For larger problem sizes exceeding 2^{20} points, our hierarchical decomposition strategy maintains favorable scalability by recursively breaking down large NTTs into smaller subproblems that fit within on-chip memory. Although this introduces additional data transfer overhead, our optimizations—particularly the double-buffering and computation-communication overlap techniques—effectively mitigate the associated performance costs.

The framework's design fully leverages the FT-M7032's multi-level memory hierarchy to balance computational efficiency and memory usage across different problem scales. This architectural consideration ensures that our approach remains applicable not only to current requirements but also provides a solid foundation for accommodating future advanced applications with more demanding computational characteristics.

7 Conclusion

7.1 Summary of Contributions

This paper has presented a comprehensive optimization framework for accelerating Number Theoretic Transform computations on FT-M7032 DSP architectures. Our contributions include: (1) a DSP-adaptive four-step NTT variant that eliminates memory-access bottlenecks through matrix decomposition; (2) register-resident microkernels with SIMD vectorization that significantly enhance hardware utilization; (3) a 16-way parallel vectorized Montgomery modular multiplication scheme that enables efficient modular arithmetic without division instructions; and (4) double buffering with pipelining that maximizes compute-communication overlap.

The experimental results demonstrate that our integrated approach achieves substantial performance improvements over existing methods, particularly for

cryptographically relevant problem sizes. The ablation studies provide valuable insights into the individual contributions of each optimization technique, offering guidance for future development efforts. More importantly, our work establishes a solid foundation for deploying efficient NTT implementations in resource-constrained embedded environments, thereby advancing the practical applicability of privacy-preserving technologies based on homomorphic encryption and lattice-based cryptography.

By addressing the unique challenges of DSP architectures through a co-design methodology that harmonizes algorithmic innovation with hardware-aware implementation strategies, this research contributes to the broader goal of enabling secure and efficient computation on encrypted data across diverse computing platforms.

7.2 Limitations and Future Research Directions

Despite the demonstrated effectiveness of our optimization framework, several limitations warrant attention in future research. The current implementation exhibits tight coupling with the FT-M7032 DSP architecture, necessitating significant parameter retuning when porting to other DSP or embedded platforms. Developing more architecture-agnostic optimization strategies would enhance the framework's generalizability and ease of adoption across diverse hardware environments.

Additionally, our present optimizations focus primarily on single DSP clusters. As multi-cluster configurations become increasingly prevalent in high-performance embedded systems, investigating efficient task distribution strategies across multiple clusters emerges as a critical research direction. This would involve addressing challenges related to load balancing, inter-cluster communication overhead, and coordinated resource management.

Another area for improvement concerns the handling of large moduli, particularly near 16-bit boundaries where precision and overflow issues become more pronounced. Refining the mathematical foundations and implementation strategies for these edge cases would enhance the robustness and applicability of our approach across a wider range of cryptographic parameters.

Future work should also explore the integration of machine learning techniques for automated parameter tuning and optimization strategy selection, potentially enabling more adaptive and self-optimizing NTT implementations. Furthermore, extending our methodology to support emerging homomorphic encryption schemes and post-quantum cryptographic standards would amplify its impact on privacy-preserving computation and secure data processing.

Acknowledgments

This work was partially supported by the National Key Research and Development Program of China (Nos. 2023YFB3002300, 2021YFB0300101), the National Natural Science Foundation of China (Grant Nos. 61902413, 12102468, 12002380), and the National University of Defense Technology Foundation (Nos. ZK21-02, ZK20-52).

References

- [Bailey, 90] D. H. Bailey, “FFTs in External or Hierarchical Memory,” *Journal of Supercomputing*, vol. 4, no. 1, pp. 23–35, 1990. DOI:10.1007/BF00162341.
- [Barrett, 87] P. Barrett, “Implementing the Rivest Shamir and Adleman Public Key Encryption Algorithm on a Standard Digital Signal Processor,” in *Advances in Cryptology — CRYPTO ’86*, A. M. Odlyzko, Ed., *Lecture Notes in Computer Science*, vol. 263, Springer, 1987, pp. 311–323, doi:10.1007/3-540-47721-7_24.
- [Bos, 18] J. Bos, L. Ducas, E. Kiltz, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Kyber: A CCA-Secure Module-Lattice-Based KEM,” in *Proc. 2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 353–367, doi:10.1109/EuroSP.2018.00032.
- [Cheon, 17] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” *Advances in Cryptology — ASIACRYPT 2017, Lecture Notes in Computer Science*, vol. 10624, pp. 409–437, 2017, doi:10.1007/978-3-319-70694-8_15.
- [Cooley, 65] J. W. Cooley and J. W. Tukey, “An Algorithm for the Machine Calculation of Complex Fourier Series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965, doi:10.1090/S0025-5718-1965-0178586-1.
- [Ducas, 18] L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme,” in *Proc. 2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2018, pp. 28–37, doi:10.1109/EuroSP.2018.00028.
- [Fan, 12] J. Fan and F. Vercauteren, “Somewhat Practical Fully Homomorphic Encryption,” *IACR Cryptology ePrint Archive*, Report 2012/144, 2012.
- [Frigo, 05] M. Frigo and S. G. Johnson, “The Design and Implementation of FFTW3,” *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005, doi:10.1109/JPROC.2004.840301.
- [Frigo, 99] M. Frigo, C. E. Leiserson, H. Prokop, and S. Ramachandran, “Cache-Oblivious Algorithms,” in *Proc. 40th Annual Symposium on Foundations of Computer Science (FOCS)*, 1999, pp. 285–298, doi:10.1109/SFCS.1999.814612.
- [Guo, 24] J. Guo, P. Liu, X. Yang, L. Zhang, and D. Wu, “Parallel Optimization of Large-Point FFT on Sunway 26010,” *J. Zhejiang Univ. (Eng. Sci.)*, vol. 58, no. 1, pp. 78–86, 2024, doi:10.3785/j.issn.1008-973X.2024.01.009.
- [Harvey, 19] D. Harvey and J. van der Hoeven, “Faster Integer Multiplication Using Short Lattice Vectors,” *The Open Book Series*, vol. 2, no. 1, pp. 293–310, 2019, doi:10.2140/obs.2019.2.293.
- [Intel Labs, 21] Intel Labs, “Intel HEXL: Homomorphic Encryption Acceleration Library,” *Software repository (GitHub)*, 2021.
- [Longa, 16] P. Longa and M. Naehrig, “Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography,” *International Conference on Cryptology & Network Security*. Springer International Publishing, 2016. DOI:10.1007/978-3-319-48965-0_8.
- [Lyubashevsky, 10] V. Lyubashevsky, C. Peikert, and O. Regev, “On Ideal Lattices and Learning With Errors over Rings,” in *Advances in Cryptology — EUROCRYPT 2010, Lecture Notes in Computer Science*, vol. 6110, Springer, 2010, pp. 1–23, doi:10.1007/978-3-642-13190-5_1.

- [Microsoft Research, 15] Microsoft Research, “Microsoft SEAL (Simple Encrypted Arithmetic Library),” Software repository (GitHub), 2015.
- [Montgomery, 85] P. L. Montgomery, “Modular Multiplication Without Trial Division,” *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985, doi:10.1090/S0025-5718-1985-0771044-8.
- [NIST, 24] National Institute of Standards and Technology (NIST), “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM),” FIPS Publication (NIST FIPS 203), Aug 2024, doi:10.6028/NIST.FIPS.203. (NIST official page / PDF)
- [Özcan, 23] A. Ş. Özcan and E. Savaş, “Two Algorithms for Fast GPU Implementation of NTT,” *IACR Cryptology ePrint Archive*, Report 2023/1410, 2023.
- [Özker, 21] Ö. Özker, C. Elgezen, A. C. Mert, E. Öztürk, and E. Savaş, “Efficient Number Theoretic Transform Implementation on GPU for Homomorphic Encryption,” *The Journal of Supercomputing*, vol. 77, no. 11, pp. 13004–13028, 2021, doi:10.1007/s11227-021-03980-5.
- [Pollard, 71] J. M. Pollard, “The Fast Fourier Transform in a Finite Field,” *Mathematics of Computation*, vol. 25, no. 114, pp. 365–374, 1971, doi:10.1090/S0025-5718-1971-0301966-0.
- [Roy, 14] S. S. Roy, F. Vercauteren, N. Mentens, D. D. Chen, and I. Verbauwhede, “Compact Ring-LWE Cryptoprocessor,” in *Cryptographic Hardware and Embedded Systems – CHES 2014*, L. Batina and M. Robshaw (eds.), *Lecture Notes in Computer Science*, vol. 8731, Springer, 2014, pp. 371–391, doi:10.1007/978-3-662-44709-3_21.
- [Shivdika, 22] K. Shivdika, G. Jonatan, E. Mora, N. Livesay, R. Agrawal, A. Joshi, J. L. Abellán, J. Kim, and D. Kaeli, “Accelerating Polynomial Multiplication for Homomorphic Encryption on GPUs,” *Proc. 2022 IEEE International Symposium on Secure and Private Execution Environment Design (SEED)*, pp. 61–72, 2022. (Preprint: arXiv:2209.01290)
- [Takahashi, 22] D. Takahashi, “An Implementation of Parallel Number-Theoretic Transform Using Intel AVX-512 Instructions,” in *Computer Algebra in Scientific Computing (CASC 2022)*, F. Boulier et al. (eds.), *Lecture Notes in Computer Science*, vol. 13366, Springer, 2022, pp. 318–332, doi:10.1007/978-3-031-14788-3_18.
- [van der Hoeven, 14] J. van der Hoeven, “Modular SIMD Arithmetic in Mathemagix,” *Acm Transactions on Mathematical Software*, 2014. DOI:10.1145/2876503.
- [Wang, 23] C. Wang and M. Gao, “SAM: A Scalable Accelerator for Number Theoretic Transform Using Multi-Dimensional Decomposition,” in *2023 IEEE/ACM Int. Conf. on Computer-Aided Design (ICCAD)*, 2023, pp. 1–9, doi:10.1109/ICCAD57390.2023.10323744.
- [Yin, 22] S. F. Yin, Q. L. Wang, J. Liu, et al., “Optimizing Irregular-Shaped Matrix-Matrix Multiplication on Multi-Core DSPs,” *Proc. IEEE Int. Conf. on Cluster Computing (CLUSTER)*, 2022, pp. 451–461. (Preprint / arXiv:2208.05872)
- [Yu, 24] K. N. Yu, X. X. Qi, Z. Wang, et al., “Optimizing General Matrix Multiplications on Modern Multi-Core DSPs,” in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2024, pp. 964–975.
- [Zhao, 23] Y. Zhao, F. Liu, W. Ma, H. Li, Y. Peng, and C. Wang, “MFFT: A GPU Accelerated Highly Efficient Mixed-Precision Large-Scale FFT Framework,” *ACM Trans. Archit. Code Optim.*, vol. 20, no. 3, Article 43, 2023, doi:10.1145/3605148.